COLLEGE OF INFORMATION AND INFORMATION TECHNOLOGY

TO: **MS MARY KADWA**

FROM**:** **HOPE KHOLOWA**

PROGRAM: ICT

COURSE CODE: 2420017

SEMESTER: 1

ASSIGNMENT**:** **2D ARRAY ELEMENTS USING POINTER ADDRESSING IN C**

DUE DATE: APRIL 17, 2026

1. Introduction

Arrays are one of the most important data structures in the C programming language. They are used to store multiple values of the same data type in contiguous memory locations. A two-dimensional array is commonly used to represent data in rows and columns, such as tables or matrices.

This assignment demonstrates how to print elements of a two-dimensional array using two different approaches:

1. Standard array indexing notation
2. Pointer/address notation

The purpose is to show the relationship between arrays and pointers in C and to understand how multidimensional arrays are stored and accessed in memory.

2. Objectives

The objectives of this assignment are:

- Rewrite the given C program correctly
- Print 2D array elements using pointer arithmetic
- Print the same elements using normal indexing
- Compare outputs from both methods
- Demonstrate understanding of memory addressing in arrays
- Improve programming documentation and code presentation

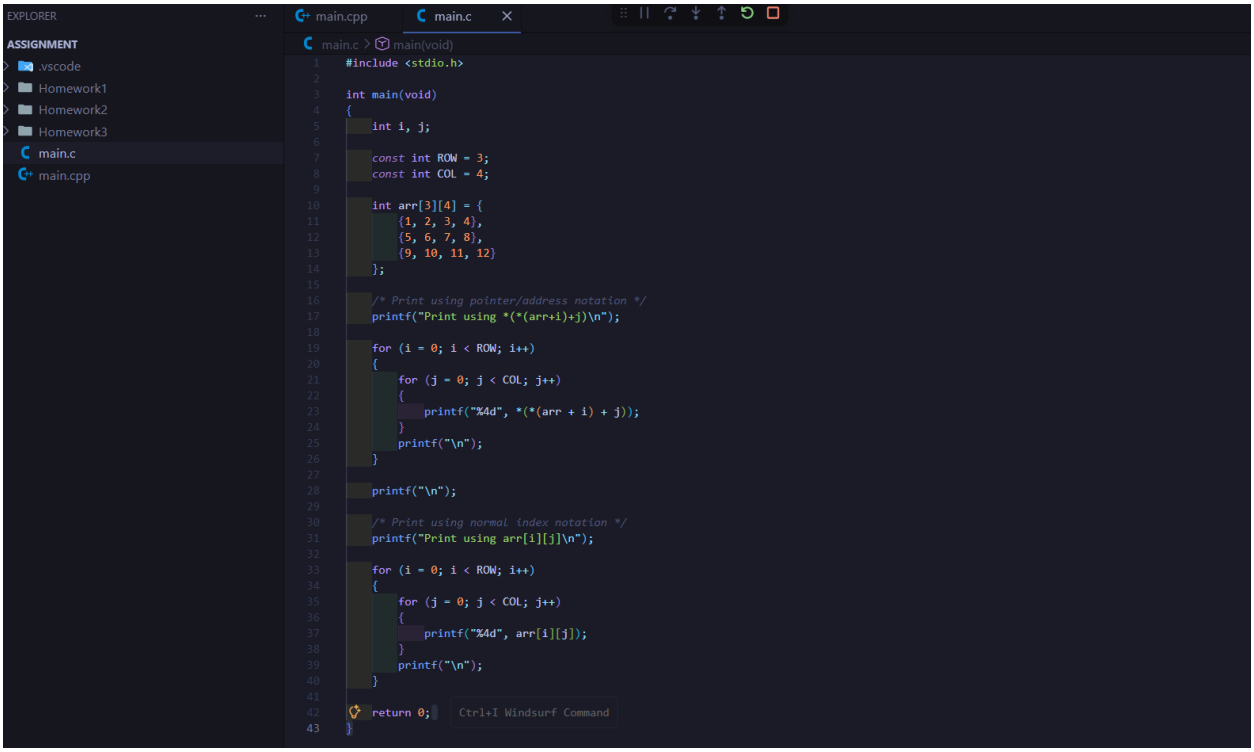
3. Problem Statement

Write a C program that stores integer values in a two-dimensional array and prints all elements using:

- Pointer/address notation: *(*(arr+i)+j)
- Standard indexing notation: arr[i][j]

Both methods must produce identical output.

CODE



5. Code Explanation

5.1 Header File

#include <stdio.h>

This library is required for input and output operations such as printf().

5.2 Main Function

int main(void)

This is the starting point of program execution.

5.3 Variable Declaration

int i, j;

These variables are loop counters:

- i represents rows
- j represents columns

5.4 Row and Column Size

const int ROW = 3;
const int COL = 4;

Defines array dimensions:

- 3 rows
- 4 columns

5.5 Array Declaration

int arr[3][4]

This creates a two-dimensional integer array containing:

Row	Values
0	1 2 3 4
1	5 6 7 8
2	9 10 11 12

5.6 Nested Loops

Used to move through every element of the array.

Outer loop = rows
Inner loop = columns

5.7 Standard Indexing

arr[i][j]

This accesses the value at row i and column j.

Example:

arr[1][2] = 7

5.8 Pointer/Address Notation

((arr + i) + j)

This is equivalent to arr[i][j].

Breakdown:

Step 1:

arr

Represents base address of first row.

Step 2:

arr + i

Moves to row i.

Step 3:

*(arr + i)

Gets the selected row.

Step 4:

*(arr + i) + j

Moves to column j.

Step 5:

((arr + i) + j)

Retrieves actual element value.

5.9 Why Outputs Are Same

Because:

arr[i][j] == *(*(arr+i)+j)

They are two different ways of accessing the same memory location.

CONSOLE OUTPUT

```
Print using *(*(arr+i)+j)
   1   2   3   4
   5   6   7   8
   9  10  11  12

Print using arr[i][j]
   1   2   3   4
   5   6   7   8
   9  10  11  12
```

Conclusion

This assignment successfully demonstrated two methods of accessing elements in a two-dimensional array in C. Both standard indexing and pointer arithmetic produced identical results. This proves that arrays and pointers are closely related in memory operations. Understanding pointer notation is important for advanced programming, memory management, and efficient data manipulation in C.

REFERENCE

https://github.com/kaymonade/C-array-pointer.git